面向长期对话的动态记忆管理与证据问答系统

摘要：长期对话中的人物状态随时间变化，检索到的相关信息需要结合问题时间判断其适用性。本文实现一套动态记忆管理与证据问答系统，以完整 Build 绑定原始来源、结构化 Memory、历史关系和索引，通过 Hybrid 检索或 Dynamic 解析生成带引用的回答。系统提供 Memory 时间线、来源跳转和分阶段 Trace，并支持独立离线回放。实验分为历史四策略基线、M5 开发与留出评估，以及 Sidecar 真实诊断研究。M5 留出实验中，Dynamic 的类别 1-4 完整分母 F1 比 Hybrid 高 0.006545，95% 区间跨零，已知 usage 对应的查询峰价估算约为 Hybrid 的 7.90 倍。Sidecar 在两个真实开发 Build 上的信息性正确数为 0/68。当前公共入口采用 Hybrid 默认、Dynamic 显式可选的策略，研究性扩展继续独立验证。

1 问题与系统范围

长期对话会积累同一人物在不同阶段的状态，也包含计划、否定与相互矛盾的陈述。仅依赖相似度排序或记录的新旧顺序，容易把历史事实用于当前问题，或者把计划视为已经完成的事件。系统因此同时保存来源轮次与时间条件，让证据选择能够区分当前状态和历史情境。

项目实现数据导入、原子 Memory 抽取、版本化构建和证据问答，并保存成功及失败 Trace 供复查。当前检索与证据处理面向英文，交互界面采用中文。构建输入使用 conversation，QA 与标注进入独立评测流程。原始事实、关系判断及当前状态分别维护，CurrentState 作为可重算投影使用。

截至 2026 年 11 月 8 日，M5 核心通过 PR #40 进入 main，本文对应公开提交 0ebb2b8。Product Console 增量位于独立分支 44d20dc，Sidecar 诊断研究位于 2deefd5。本文分别报告三部分的实现和证据，Console 与 Sidecar 的最新成果仍待跨分支集成验证。

2 架构、时间语义与默认策略

系统采用 Streamlit、FastAPI 与 SQLite 的分层结构。SourceTurn 以 dataset、sample 和 dia_id 标识来源，Memory 可以关联多个原始轮次。Build 依次完成抽取与校验，生成检索索引和历史关系，再发布同一版本的状态投影。片段任务保存检查点，重试复用成功产物，失败版本保持可检查状态。

查询固定一个完整 Build。Hybrid 通过 BM25 与 Dense 检索产生候选，使用 RRF 融合排序，并执行有界历史扩展后生成回答。Dynamic 在此基础上增加有效证据解析，记录选择理由及补充判断。查询期的补判写入本次 Trace，全局 CurrentState 保持构建版本的身份。回答同时返回引用，便于从结果定位到原始证据。

时间模型分别保留会话报告时刻与事件时间。当前查询采用对话的证据截止时间，指定日期的查询采用对应历史条件。SUPERSEDE 更新当前状态中的旧取值，旧证据继续支持历史问题；CONFLICT 保留同一适用条件下尚待消解的矛盾。计划和否定沿用相应状态，避免通过时间排序直接推导事实成立。

main 的公共页面和省略 strategy 的 POST /queries 默认使用 Hybrid，Dynamic 由用户显式选择[2]。Graph 多跳、压缩 wire、自适应路由及新增时间软优先均默认关闭，既有关系、版本和时间能力保留，只读图浏览等展示功能可选。M5 留出实验的高输入限额属于冻结研究配置，应用继续采用自身 Settings，二者分别记录版本和预算。

3 Product Console 与离线回放

Console 增量已提供 Memory Timeline、来源跳转和四阶段 Trace。Timeline 按 API Memory 记录顺序展示，将报告时间、事件时间及有效时间分列，保留 plan、negative 和 unknown 状态。选择引用后，界面读取完整 SourceTurn，并核对来源身份。Trace 分开呈现初始候选、历史扩展、有效 Memory 和最终交付证据，错误与预算信息随记录保留。

实验比较支持按策略、对话或题型查看冻结聚合，同时展示完整分母和成功子集。独立 Replay 读取三个经过哈希校验的 JSON，支持两个 Build 切换，运行时仅需 Streamlit。六个合成案例覆盖当前回答、历史回答、冲突、证据不足，以及引用异常和预算失败，用于检查应用行为。

该分支的定向回归为 25 项通过，产品运行代码的 Linux CI 为 599 passed、6 deselected、1 warning。main 公开快照另有 967 passed、28 deselected 的检查记录[4]。两组工程测试对应各自代码范围。

4 历史四策略基线

历史 Run a2abe1fb49be45ccba291728e1f6b3c9 固定 conv-26、conv-30、conv-41、conv-42 的 757 题，每题执行四种策略，共 3,028 项。原记录含 2,969 项成功、56 项失败和 3 项 unknown[1]。Full Text 使用完整原始对话，Vector 使用向量检索，Hybrid 增加混合检索与历史扩展，Dynamic 继续解析有效证据。

表 1 历史四策略的问答结果与技术成功率

策略	计划	成功	失败	unknown	成功子集 QA	技术成功率
Full Text	757	756	0	1	0.465484	99.87%
Vector	757	751	5	1	0.523866	99.21%
Hybrid	757	753	3	1	0.524659	99.47%
Dynamic	757	709	48	0	0.550008	93.66%

双方成功的 705 对上，Dynamic 和 Hybrid 的 QA 均值分别为 0.550374、0.524300，均差 +0.026074。技术失败记零后，Dynamic 的 757 题完整分母得分为 0.515133；Hybrid 的 1 项 unknown 分别按 0 和 1 计分，区间为 [0.521886, 0.523207]。成功子集的配对收益需要与完整运行的可靠性共同解释。

根据已知 usage 和当时冻结峰价，Dynamic 与 Hybrid 的查询费用估算分别为 98.930522、9.597679 CNY，其中 Dynamic 的 Resolution 为 96.285551 CNY，占其查询估算的 97.33%。共享 Build 的已知 usage 峰价估算为 89.655017 CNY，未知 usage 预留为 7.077888 CNY；Hybrid 查询另有 2.359296 CNY 未知预留。这些估算与预留分别记账，均区别于账户实扣金额。

5 M5 开发与留出评估

M5 进一步比较 Hybrid 与 Dynamic。开发 input v2 使用同一组 100 题，其中 80 题为普通问题、20 题为诊断问题。普通问题的完整分母得分分别为 0.401172、0.444702，差值为 +0.043530，95% 区间为 [-0.008095, 0.095318]。全部 100 题中，Hybrid 成功 99 项，Dynamic 成功 100 项。该版同时调整输入限额和 thinking 设置，开发阶段的变化应结合完整配置解释。

Level E 在读取 Gold 前固定配置并冻结生成结果，对 conv-43、conv-44 的 400 题分别运行两臂，共 800 项，Bonus 关闭[3]。其中类别 1-4 每臂 301 题，采用 token F1，失败记零。Hybrid 为 0.402606，Dynamic 为 0.409151，配对差为 +0.006545，95% 按题 bootstrap 区间为 [-0.020133, 0.034035]。区间跨零，当前样本尚不足以确立稳定的质量改善。

类别 5 的 99 题单独采用字面拒答规则，Hybrid 为 0.858586，Dynamic 为 0.808081。这一指标反映特定拒答表达的命中情况，与类别 1-4 的事实问答分开解读。留出结果现已曝光，后续在相同问题上的修复与重跑作为回归；按题重采样的区间描述这两段对话内的题目不确定性。

全部 800 项另有来源盲审，每项由一名非作者 AI 审阅。来源支持分别为 Hybrid 250/343、Dynamic 233/307；双方可判断的 297 对净改善 10 对，区间仍跨零。这些分母排除了无法判断和不适用项，属于条件比较。该参考用于诊断来源支持，人工语义复核作为后续工作。

6 可靠性、费用与证据损失

Level E 中 Hybrid 成功 394/400，Dynamic 成功 389/400。17 个失败项包括 8 个 invalid_output 和 9 个 HTTP 402 余额拒绝项。失败保留并记零，技术成功率分别为 98.50% 和 97.25%。运行失败、回答拒答与来源不足分别记录，便于定位系统问题。

本轮查询共 2,112 次物理请求。已知 usage 对应的峰价估算分别为 Hybrid 0.8705958 USD、Dynamic 6.8800119 USD，后者约为前者的 7.9026 倍。18 次缺失 usage 的请求仍保留完整预约，合计 0.9522705 USD。含失败项的单题耗时 p50 分别为 2.370230 秒和 8.314747 秒，Dynamic 在本轮串行运行中具有更高的费用与延迟。

共享 Build 的已知 usage 峰价估算为 6.6141915 USD。比较某一策略的独立部署成本时，该策略需要承担完整 Build 成本，再按问题数摊销。历史实验使用 CNY，本轮使用 USD，费用均按各自冻结价格估算。

来源覆盖分析显示，Dynamic 在解析后的有效集合与最终交付阶段损失了部分标注来源。每臂 399/400 题具有可映射来源，1 题保持 null；K 表示 Memory 排名，覆盖按唯一 SourceTurn 计算。该指标有助于定位证据在哪个阶段减少，事实是否仍适用需要结合来源语义判断。

历史研究也提供了采用依据。E1 路由在 757 项历史模拟中全部选择 Dynamic，配对增益为 0。E5 的隔离基准将冷编码和本地检索 CPU 分别降低 64.22%、57.36%，收益限于本地计算，provider token 与费用节省为 0。M5 的 Graph 扩展、压缩和自适应模块保留工程能力，现有回放结果支持继续采用默认关闭的策略。

7 Sidecar 真实诊断研究

Sidecar 将陈述时间 reported_at 与事件时间 event_time 分开建模，并绑定 Build fingerprint、Memory 与来源 ID。11 月 7 日的独立研究使用 deterministic-sidecar-v0.5-research，对 conv-26、conv-30 两个冻结的 MemoryExportV2 导出进行诊断，共 503 条 Memory、344 个来源及 63,057 对候选，输入完整且无截断[5]。

两个 Build 的时间、完成和出席判断全部为 unknown，事件关系判断也全部为 unknown。失败分析中，502 条 Memory 超出规则支持的断言语法，另 1 条缺少合格匹配引用。主要瓶颈是合成句式与真实 Memory 表达之间的适配。

研究固定 40 条 Memory、20 个事件对和 8 个状态问题，共 68 个参考单元，由非作者 AI 各进行一次参考标注。三个 Memory 判断维度共享同一组 40 条记录。只读字段控制组与 Sidecar 的信息性正确数均为 0/68。8 个状态问题均返回 insufficient，其中 1 项与 sufficient 参考不一致，其余 7 项的参考为 unknown。保守的证据不足输出仍可能遗漏可回答的信息。

控制组 baseline_fields 采用只读字段，其任务范围区别于 Hybrid 问答；所用数据已经曝光，结论定位于开发诊断。研究新增外部 Provider 请求为 0，开关前后保持原导出与响应不变。当前建议保留默认关闭的离线诊断模式。下一候选应先验证来源绑定与真实句式覆盖，再评估其对答案的影响。

8 结论与后续工作

系统已形成从原始轮次到 Memory、固定 Build 和引用回答的处理链路，Console 将关键证据与失败记录呈现在界面中。现有实验表明，Dynamic 的局部质量收益伴随更高的解析成本，留出结果仍缺少稳定的总体改善证据。因此公共入口采用 Hybrid 默认、Dynamic 显式可选，研究性扩展独立保留。

后续工作集中于证据选择与运行可靠性：结合来源逐项分析有效集合中的证据损失，补充人工语义标签，并完成 Console 与公共 API 默认策略的集成回归。Sidecar 继续以真实表达覆盖为前置目标，采用判断由完整分母效果、可靠性与代价共同支持。

9 参考资料

[1] 历史四策略聚合：docs/team-v2/research-evidence.json，原 Run a2abe1fb49be45ccba291728e1f6b3c9。

[2] M5 核心：main 0ebb2b8，PERSONAL_EXPERIMENT_SUMMARY.md、RELEASE_COMPONENT_MATRIX.md 和 docs/release/DEFAULTS_AND_ROLLBACK.md。

[3] M5 留出结果：docs/m5-development/LEVEL_E_RESULTS_V1.md，生成提交 0dbd98e；分数与来源参考位于 experiments/m5-development/level-e-results-v1。

[4] Console 增量：44d20dc，docs/team-v2/product-followup-validation.md；main 工程检查见 docs/public-release/PUBLIC_SNAPSHOT_VALIDATION.json。

[5] Sidecar 研究：2deefd5，research/sidecar-20261107/REPORT.md、run-v2-summary.json、run-v2-analysis.json 和 grammar-failure-analysis.json。